



DEPARTMENT OF PHYSICS

TFY4345 - CLASSICAL MECHANICS

Circular Restricted Three Body Problem

Author:
Mathias T. Thorkildsen

Date: 20.10.2025

Table of Contents

List of Figures	i
List of Tables	i
1 Introduction	1
2 Non-Dimensional Equations of Motion	1
3 First Order Conversion & Runge-Kutta Solver	2
4 Jacobi Integral	4
5 Lagrange Points	4
Bibliography	8
Appendix	9
A CR3BP solver and example	9
B Jacobi integral calculation	12
C Collinear Lagrange points calculation	13
D Orbit calculation for initial condition at L_1	16

List of Figures

1	<i>Example plot for a planar trajectory in the rotating frame of reference.</i>	3
2	<i>Example plot for a planar trajectory in the inertial frame of reference.</i>	3
3	<i>Plot of the Jacobi integral for our chosen initial conditions</i>	4
4	<i>The collinear Lagrange points obtained for our Earth-Moon primary system using Newton's method.</i>	6
5	<i>Orbit with initial condition at rest at L_1 in the rotating frame.</i>	6
6	<i>Orbit with initial condition at rest at L_1 in the inertial frame.</i>	7

List of Tables

1	<i>Computed positions and iteration counts for the collinear Lagrange points in the CR3BP (Earth-Moon system, $\mu = 0.01215$).</i>	5
---	--	---

1 Introduction

This report presents an analysis of the so-called Circular Restricted Three-Body Problem (CR3BP), commonly used in orbital dynamics to model the motion of small bodies, such as satellites or comets around a primary system of considerably larger masses [1].

From the theoretical introduction given in the assignment, we have a set of equations of motion for our satellite moving with the primary system in a rotating frame of reference,

$$\ddot{x} = 2\omega\dot{y} + \omega^2 x - \frac{a^3\omega^2(1-\mu)(x+\mu a)}{s_1^3} - \frac{a^3\omega^2\mu(x-a+\mu a)}{s_2^3}, \quad (1)$$

$$\ddot{y} = -2\omega\dot{x} + \omega^2 y - \frac{a^3\omega^2(1-\mu)y}{s_1^3} - \frac{a^3\omega^2\mu y}{s_2^3}, \quad (2)$$

$$\ddot{z} = -\frac{a^3\omega^2(1-\mu)z}{s_1^3} - \frac{a^3\omega^2\mu z}{s_2^3}. \quad (3)$$

The primary system is assumed to exhibit a constant, circular orbit of radius a , and ω is the corresponding angular velocity.

$$\mu = \frac{m_2}{m_1 + m_2}$$

is the dimensionless relative mass of the primary system, consisting of the masses m_1 and m_2 . s_1 and s_2 are the distances between our satellite of mass m , and m_1 , m_2 respectively. The gravitational constant is also eliminated from the equations by using the relations,

$$\begin{aligned} Gm_1 &= a^3\omega^2(1-\mu), \\ Gm_2 &= a^3\omega^2\mu. \end{aligned}$$

Generally, from the Buckingham Pi theorem, we know that our fully non-dimensionalized system will contain a total of six non-dimensional variables. Therefore, we will further non-dimensionalize our equations in order to more effectively analyze it utilizing numerical methods [1].

2 Non-Dimensional Equations of Motion

To non-dimensionalize the equations of motion, we introduce the variables

$$x' = \frac{x}{a}, \quad y' = \frac{y}{a}, \quad z' = \frac{z}{a}, \quad \tau = \omega t.$$

This introduces a linear transformation

$$T : (\mathbf{r}, t) \mapsto (\mathbf{r}', \tau),$$

where $\mathbf{r} = (x, y, z)^T$ and $\mathbf{r}' = (x', y', z')^T$. Explicitly, the coordinate part of the transformation is given by

$$\mathbf{r}' = \frac{1}{a} I_3 \mathbf{r}.$$

Applying T to the linear operators for the time derivatives gives

$$\begin{aligned} \frac{d}{dt} &= \frac{d\tau}{dt} \frac{d}{d\tau} = \omega \frac{d}{d\tau}, \\ \frac{d^2}{dt^2} &= \omega \frac{d}{d\tau} \left(\omega \frac{d}{d\tau} \right) = \omega^2 \frac{d^2}{d\tau^2}. \end{aligned}$$

Thus, T can be represented as a linear transformation on the extended space of coordinates and derivative operators by

$$\begin{pmatrix} \mathbf{r}' \\ \frac{d}{d\tau} \end{pmatrix} = \begin{pmatrix} \frac{1}{a}I_3 & 0 \\ 0 & \frac{1}{\omega} \end{pmatrix} \begin{pmatrix} \mathbf{r} \\ \frac{d}{dt} \end{pmatrix}.$$

Applying T to s_1 and s_2 gives

$$\begin{aligned} T(s_1) &= \sqrt{(ax' + \mu a)^2 + (ay')^2 + (az')^2} = as'_1, \\ T(s_2) &= \sqrt{(ax' - a + \mu a)^2 + (ay')^2 + (az')^2} = as'_2, \end{aligned}$$

with s'_1, s'_2 defined as given in the problem set ([1]).

We can then apply T to equations (1), (2), and (3), and obtain

$$\begin{aligned} a\omega^2 \dot{x}' &= 2a\omega^2 \dot{y}' + a\omega^2 x' - \frac{a^4\omega^2(1-\mu)(x' + \mu)}{a^3 s_1'^3} - \frac{a^4\omega^2\mu(x' - 1 + \mu)}{a^3 s_2'^3}, \\ a\omega^2 \dot{y}' &= -2a\omega^2 \dot{x}' + a\omega^2 y' - \frac{a^4\omega^2(1-\mu)y'}{a^3 s_1'^3} - \frac{a^4\omega^2\mu y'}{a^3 s_2'^3}, \\ a\omega^2 \dot{z}' &= -\frac{a^4\omega^2(1-\mu)z'}{a^3 s_1'^3} - \frac{a^4\omega^2\mu z'}{a^3 s_2'^3}, \end{aligned}$$

where we use dot notation for the derivative with respect to τ .

Simplifying the fractions and dividing the equations by $a\omega^2$, we get

$$\ddot{x}' = 2\dot{y}' + x' - \frac{(1-\mu)(x' + \mu)}{s_1'^3} - \frac{\mu(x' - 1 + \mu)}{s_2'^3}, \quad (4)$$

$$\ddot{y}' = -2\dot{x}' + y' - \frac{(1-\mu)y'}{s_1'^3} - \frac{\mu y'}{s_2'^3}, \quad (5)$$

$$\ddot{z}' = -\frac{(1-\mu)z'}{s_1'^3} - \frac{\mu z'}{s_2'^3}. \quad (6)$$

These are the non-dimensional equations of motion governing the CR3BP. They involve six variables, in accordance with the Buckingham Pi theorem [1].

3 First Order Conversion & Runge-Kutta Solver

To facilitate numerical analysis of the system, we convert the three second-order equations (4), (5), (6), into a set of six first-order equations,

$$\begin{aligned} \dot{x}' &= \xi, \\ \dot{y}' &= \eta, \\ \dot{z}' &= \zeta, \\ \dot{\xi} &= 2\eta + x' - (1-\mu)\frac{x' + \mu}{s_1'^3} - \mu\frac{x' - 1 + \mu}{s_2'^3}, \\ \dot{\eta} &= -2\xi + y' - (1-\mu)\frac{y'}{s_1'^3} - \mu\frac{y'}{s_2'^3}, \\ \dot{\zeta} &= -(1-\mu)\frac{z'}{s_1'^3} - \mu\frac{z'}{s_2'^3}. \end{aligned}$$

Following the assignment's recommendation, we employ the 8th-order Runge-Kutta method given by **DOP853** in **scipy.integrate.solve_ivp** [1]. An example result, resembling that of an Earth-Moon satellite, is shown in figure 1. In this example, we therefore choose $\mu = 1.215 \cdot 10^{-2}$. The integration starts in low Earth orbit at $r_{\text{phys}} = 6600$ km, corresponding to an altitude of approximately 222 km. To obtain a translunar trajectory, the initial velocity after the injection was set to $\eta = 10.7$, using data from the Apollo 11 mission report [2]. Making use of the equation

$$\begin{pmatrix} x_I \\ y_I \\ z_I \end{pmatrix} = \begin{pmatrix} \cos(\omega t) & -\sin(\omega t) & 0 \\ \sin(\omega t) & \cos(\omega t) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_R \\ y_R \\ z_R \end{pmatrix},$$

a function was implemented to transform from the rotating to the inertial frame of reference. The results of this transformation can be viewed in figure 2. The code used in this entire section can be found in appendix A.

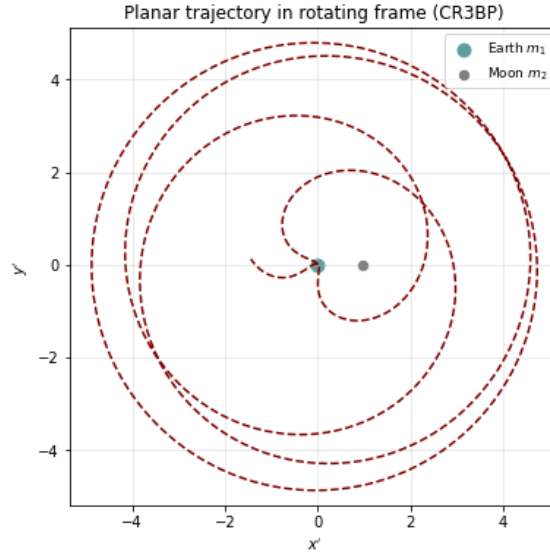


Figure 1: *Example plot for a planar trajectory in the rotating frame of reference.*

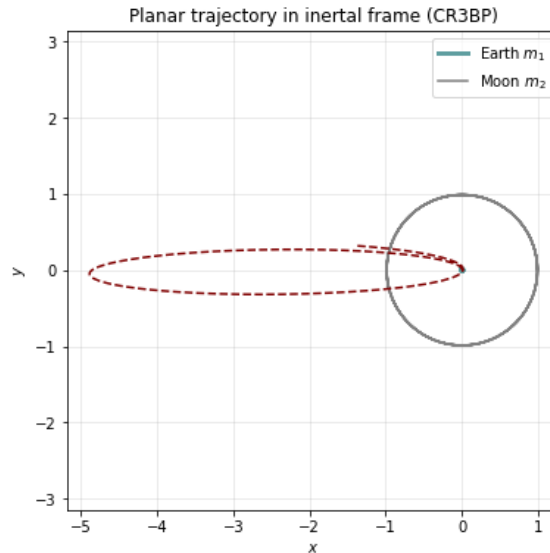


Figure 2: *Example plot for a planar trajectory in the inertial frame of reference.*

4 Jacobi Integral

To verify the physical accuracy of our solver, we will calculate the Jacobi integral

$$C(\mathbf{r}, \dot{\mathbf{r}}) = -(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) - 2\tilde{U}, \quad (7)$$

with

$$\tilde{U} = -\frac{1}{2}(x^2 + y^2) - \frac{1-\mu}{s_1} - \frac{\mu}{s_2} - \frac{1}{2}\mu(1-\mu) \quad (8)$$

as given in the assignment [1]. This is a constant of motion, and should therefore remain constant throughout the simulation. The results of this calculation can be viewed in figure 3, and the code used to obtain this result is displayed in appendix B. As we observe in the figure, the difference between the minimum and maximum values of the integral is on the 10^{-8} scale. Such small deviations are to be expected from our numerical method, and confirm our solver is working as intended.

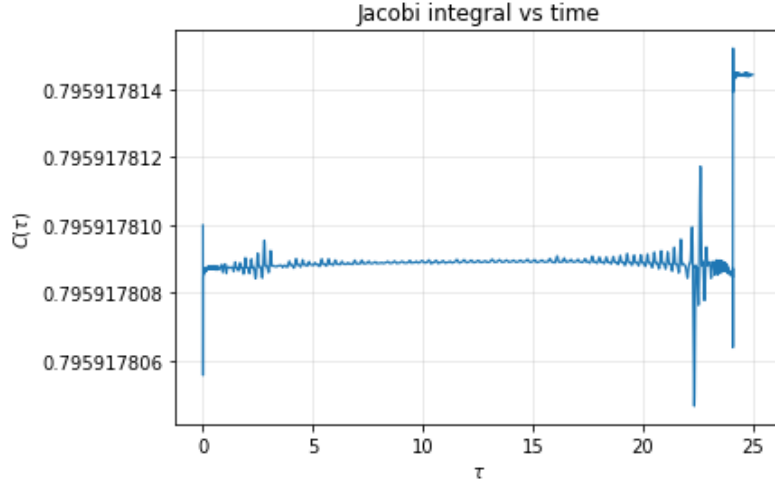


Figure 3: Plot of the Jacobi integral for our chosen initial conditions

5 Lagrange Points

We now analyze the case where $y = 0$, and find the critical points along the x-axis. These are the so-called collinear Lagrange points L_1 , L_2 , and L_3 [1]. To find these points, we will extremize the function

$$\tilde{U}(x, 0) = -\frac{1}{2}x^2 - \frac{1-\mu}{|x+\mu|} - \frac{\mu}{|x-1+\mu|} - \frac{1}{2}\mu(1-\mu).$$

In other words, we want to find the points where $\partial_x \tilde{U}(x, 0) = 0$. Given the absolute values present in the equation, we have three different domains to solve over; $x \in (-\infty, -\mu) = I_3$, $x \in [-\mu, 1-\mu] = I_1$, and $x \in (1-\mu, \infty) = I_2$. Differentiation over the different domains yields

$$\partial_x \tilde{U}(x, 0) = \begin{cases} -x - \frac{1-\mu}{(x+\mu)^2} - \frac{\mu}{(x-1+\mu)^2}, & x < -\mu, \\ -x + \frac{1-\mu}{(x+\mu)^2} - \frac{\mu}{(x-1+\mu)^2}, & -\mu \leq x \leq 1-\mu, \\ -x + \frac{1-\mu}{(x+\mu)^2} + \frac{\mu}{(x-1+\mu)^2}, & x > 1-\mu. \end{cases} \quad (9)$$

From the Abel-Ruffini theorem, we know that the quintic equation $\partial_x \tilde{U}(x, 0) = 0$ exhibits no closed form solution [3]. We therefore employ Newton's method to approximate the solutions for the three different cases.

Newton's method is an iterative numerical scheme used to find roots of a nonlinear scalar equation

$$f(x) = 0.$$

Starting from an initial guess x_0 , the method constructs a sequence

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},$$

which converges quadratically to the true root x^* if f is sufficiently smooth and the initial guess is close enough to the solution.

In practice, the iteration continues until the relative error

$$\left| \frac{x_{n+1} - x_n}{x_{n+1}} \right| < \varepsilon$$

falls below a chosen tolerance ε .

For the present problem, Newton's method is particularly convenient since both $f(x) = \partial_x \tilde{U}(x, 0)$ and its derivative $f'(x)$ can be computed analytically, leading to rapid convergence to the collinear Lagrange points L_1, L_2, L_3 .

Differentiating equation (9) gives

$$\partial_{xx} \tilde{U}(x, 0) = \begin{cases} -1 + \frac{2(1-\mu)}{(x+\mu)^3} + \frac{2\mu}{(x-1+\mu)^3}, & x < -\mu, \\ -1 - \frac{2(1-\mu)}{(x+\mu)^3} + \frac{2\mu}{(x-1+\mu)^3}, & -\mu \leq x \leq 1-\mu, \\ -1 - \frac{2(1-\mu)}{(x+\mu)^3} - \frac{2\mu}{(x-1+\mu)^3}, & x > 1-\mu. \end{cases} \quad (10)$$

Using equations (9) and (10) Newton's method was then implemented, and the resulting Lagrange points for our Earth-Moon-like primary system can be found in table 1, and visualized in figure 4. The code used to obtain these results can be found in appendix C.

Lagrange Point	x' -position	Iterations
L_1	0.8369180073169304	7
L_2	1.1556799130947353	12
L_3	-1.0050624018204988	16

Table 1: *Computed positions and iteration counts for the collinear Lagrange points in the CR3BP (Earth-Moon system, $\mu = 0.01215$).*

Lastly, we calculate an orbit with initial condition at rest at L_1 . The result of which is shown in figures 5 and 6, displayed in the rotating and inertial frame respectively. As observed, the trajectories diverge away from L_1 after some time, due to tiny fluctuations in position. This shows us that the point L_1 is an unstable equilibrium.

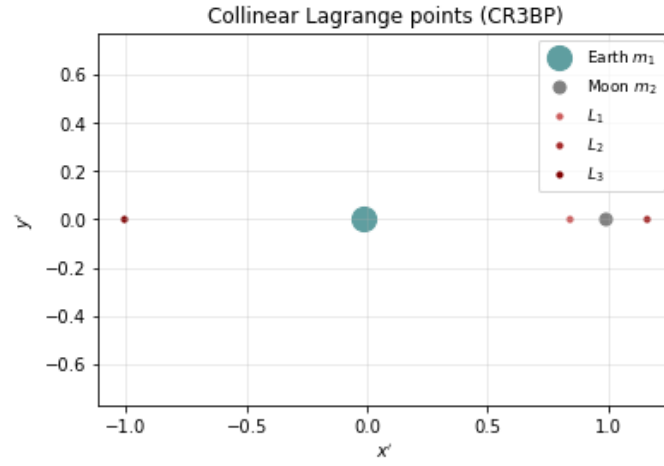


Figure 4: *The collinear Lagrange points obtained for our Earth-Moon primary system using Newton's method.*

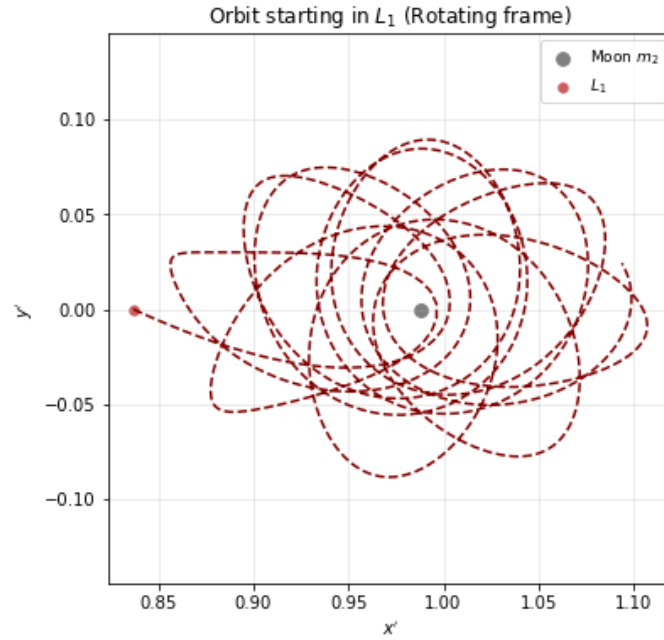


Figure 5: *Orbit with initial condition at rest at L_1 in the rotating frame.*

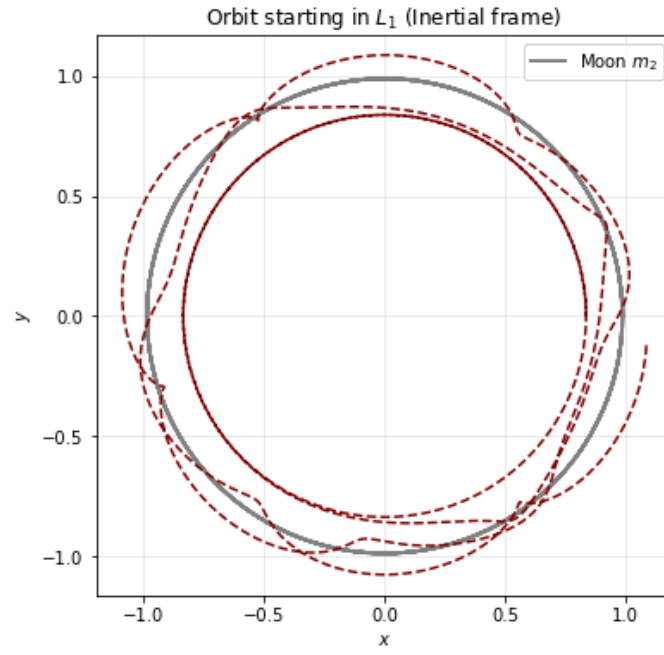


Figure 6: *Orbit with initial condition at rest at L_1 in the inertial frame.*

Bibliography

- [1] Markus Hamre. ‘TFY4345 Assignment Fall 2025 (CR3BP)’. In: *NaN* (2025), pp. 1–6.
- [2] NASA. *Mission Report : Apollo 11*. 1996. URL: https://www.nasa.gov/history/alsj/a11/A11_PAOMissionReport.html.
- [3] Paul Ramond. ‘The Abel–Ruffini Theorem: Complex but Not Complicated’. In: *The American Mathematical Monthly* 129.3 (Mar. 2022), pp. 231–245. ISSN: 1930-0972. DOI: 10.1080/00029890.2022.2010494. URL: <http://dx.doi.org/10.1080/00029890.2022.2010494>.

Appendix

A CR3BP solver and example

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def CR3BP_rhs(tau, X_arr):
    """
    Computes the right-hand side of the non-dimensional CR3BP equations.

    Parameters
    -----
    tau      : float
               Dimensionless time variable ( $\tau = \omega t$ ).
    X_arr     : array_like, shape (6,)
               State vector  $[x, y, z, \dot{x}, \dot{y}, \dot{z}]$ , where:
               x, y, z : float
                   Position components of the small body in the rotating frame.
               xi, eta, zeta : float
                   Velocity components with respect to tau.

    Returns
    -----
    dYdtau   : ndarray, shape (6,)
               Time derivative of the state vector:
                $[dx/d\tau, dy/d\tau, dz/d\tau, d^2x/d\tau^2, d^2y/d\tau^2, d^2z/d\tau^2]$ .

    Notes
    -----
    The system models the Circular Restricted Three-Body Problem (CR3BP)
    in a rotating reference frame, using the standard non-dimensionalized
    formulation with characteristic distance 'a = 1' and angular velocity ' $\omega = 1$ '.
    """
    x, y, z, xi, eta, zeta = X_arr

    # Distance to the two primaries
    s1_3 = ((x + mu)**2 + y**2 + z**2)**(3/2)
    s2_3 = ((x - 1 + mu)**2 + y**2 + z**2)**(3/2)

    # Calculation of the second-derivatives
    xdd = 2 * eta + x - (1 - mu) * (x + mu) / s1_3 - mu * (x - 1 + mu) / s2_3
    ydd = -2 * xi + y - (1 - mu) * y / s1_3 - mu * y / s2_3
    zdd = - (1 - mu) * z / s1_3 - mu * z / s2_3

    return np.array([xi, eta, zeta, xdd, ydd, zdd])

def rotating_to_inertial(t, xR, yR, zR, omega = 1.0, is_tau = True):
    """
    Parameters
    -----
    """
```

```

t : array_like
    Time samples. If `is_tau` is True, this is  $\tau = \omega * t$  (non-dimensional
    → time).
    Otherwise it is the physical time  $t$  (seconds) and  $\omega$  is used.

xR, yR, zR : array_like
    Rotating-frame coordinates at each time sample. Must be broadcastable to
    → the shape of `t`.

omega : float, optional
    Angular rate  $\omega$  of the rotating frame. Default 1.0.

is_tau : bool, optional
    If True, interpret `t` as  $\tau$  so the rotation angle is  $\theta = \tau$ .
    If False, interpret `t` as physical time and use  $\theta = \omega * t$ .

Returns
-----
xI, yI, zI : ndarray
    Inertial-frame coordinates at the same time samples.

"""

# Ensures data is stored in numpy arrays
t = np.asarray(t)
xR, yR, zR = np.asarray(xR), np.asarray(yR), np.asarray(zR)

# Sets the rotation angle theta
theta = t if is_tau else omega * t
cos, sin = np.cos(theta), np.sin(theta)

# Implementing the effect of the rotation matrix
xI = cos * xR - sin * yR
yI = sin * xR + cos * yR
zI = zR

return xI, yI, zI

if __name__ == '__main__':

    # Parameters
    mu = 1.215e-2          # Earth{Moon mass parameter (example)
    t_span = (0.0, 25.0)  # dimensionless time interval  $\tau$  in  $[0, T]$ 
    t_array = np.linspace(t_span[0], t_span[1], 5000)

    r_phys = 6600          # Radius of our LOE [km]
    a = 3.850e5            # Given a-value for the earth and moon [km]
    r_nd = r_phys / a      # Non-dimensionalized radius for LOE

    # Example Initial conditions (planar example:  $z = \zeta = 0$ )
    # Starting in low earth orbit, placing the satellite along the  $x'$  axis.
    x0, y0, z0 = -mu + r_nd, 0.0, 0.0
    xi0, eta0, zeta0 = 0.0, 10.7, 0.0
    Y0 = np.array([x0, y0, z0, xi0, eta0, zeta0])

```

```

# Integrate with DOP853
sol = solve_ivp(
    CR3BP_rhs, t_span, Y0, t_eval=t_array,
    method="DOP853", rtol=1e-10, atol=1e-12)

if not sol.success:
    raise RuntimeError(sol.message)

# Storing solutions for the planar motion
x = sol.y[0]
y = sol.y[1]
z = sol.y[2]

# Positions of primaries in this normalization
m1_pos = (-mu, 0.0)
m2_pos = (1 - mu, 0.0)

# Plotting planar motion
plt.figure(figsize=(6,6))
plt.plot(x, y, linestyle = '--', color = 'maroon')
plt.scatter(*m1_pos, s=80, marker='o', label='Earth $m_1$', color =
    ↪ 'cadetblue')
plt.scatter(*m2_pos, s=40, marker='o', label='Moon $m_2$', color = 'grey')
plt.axis('equal')
plt.xlabel("$x$")
plt.ylabel("$y$")
plt.title("Planar trajectory in rotating frame (CR3BP)")
plt.legend(fontsize = 9)
plt.grid(True, alpha=0.3)
plt.show()

# Applying the rotation matrix to our planar motion example
xI, yI, zI = rotating_to_inertial(t_array, x, y, z)

# Calculating the orbits of the primaries
x1_orbit = -mu * np.cos(t_array)
y1_orbit = -mu * np.sin(t_array)
x2_orbit = (1 - mu) * np.cos(t_array)
y2_orbit = (1 - mu) * np.sin(t_array)

plt.figure(figsize=(6,6))
plt.plot(x1_orbit, y1_orbit, label='Earth $m_1$', color = 'cadetblue',
    ↪ linewidth = 3)
plt.plot(x2_orbit, y2_orbit, label='Moon $m_2$', color = 'grey')
plt.plot(xI, yI, linestyle = '--', color = 'maroon')
plt.axis('equal')
plt.xlabel("$x$")
plt.ylabel("$y$")
plt.title("Planar trajectory in inertial frame (CR3BP)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

B Jacobi integral calculation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from CR3BP_solver import CR3BP_rhs, rotating_to_inertial

def effective_potential_tilde(x, y, z, mu):
    """
    Calculates the effective potential in the CR3BP

    Parameters
    -----
    x, y, z : array_like
        Position coordinates of the third body in the rotating frame. Can be
        ↪ scalars or NumPy arrays.

    mu : float
        Dimensionless mass parameter mu.

    Returns
    -----
    U_tilde : ndarray or float
        The effective potential  $\tilde{U}(x, y, z)$  evaluated at the given
        ↪ coordinates.
    """
    s1 = np.sqrt((x + mu)**2 + y**2 + z**2)
    s2 = np.sqrt((x - 1 + mu)**2 + y**2 + z**2)
    return -(0.5*(x**2 + y**2)) - (1 - mu)/s1 - mu/s2 - 0.5*mu*(1 - mu)

def jacobi_constant_from_state(Y, mu):
    """
    Calculates the Jacobi integral for a given state of the CR3BP

    Parameters
    -----
    Y : array_like, shape (6, N)
        State vector(s) in the rotating frame:
        [x, y, z, xi, eta, zeta], where (xi, eta, zeta) are the time derivatives
        ↪ of (x, y, z).
        The function accepts either a single state (6,) or multiple states (6, N)
        ↪ as used in solve_ivp.

    mu : float
        Dimensionless mass parameter mu.

    Returns
    -----
    C : ndarray or float
        Jacobi integral value(s) corresponding to each state in Y.
    """
    x, y, z, xi, eta, zeta = Y
    U_tilde = effective_potential_tilde(x, y, z, mu)
    v2 = xi**2 + eta**2 + zeta**2
    return -(v2) - 2*U_tilde
```

```

# Parameters
mu      = 1.215e-2      # Earth{Moon mass parameter (example)
t_span = (0.0, 25.0)    # dimensionless time interval tau \in [0, T]
t_array = np.linspace(t_span[0], t_span[1], 5000)

r_phys = 6600           # Radius of our LOE [km]
a       = 3.850e5        # Given a-value for the earth and moon [km]
r_nd    = r_phys / a    # Non-dimensionalized radius for LOE

# Example Initial conditions (planar example: z = zeta = 0)
# Starting in low earth orbit, placing the satellite along the x' axis.
x0, y0, z0 = -mu + r_nd, 0.0, 0.0
xi0, eta0, zeta0 = 0.0, 10.7, 0.0
Y0 = np.array([x0, y0, z0, xi0, eta0, zeta0])

# Integrate with DOP853
sol = solve_ivp(
    CR3BP_rhs, t_span, Y0, t_eval=t_array,
    method="DOP853", rtol=1e-10, atol=1e-12)

if not sol.success:
    raise RuntimeError(sol.message)

# Storing solutions for the planar motion
Y = sol.y

# Calculating the Jacobi integral and plotting the results
C = jacobi_constant_from_state(Y, mu)

plt.figure()
plt.plot(sol.t, C, linewidth=1.2)
plt.ticklabel_format(style='plain', useOffset=False)
plt.xlabel(r"$\tau$")
plt.ylabel(r"$C(\tau)$")
plt.title("Jacobi integral vs time")
plt.grid(True, alpha=0.3)
plt.show()

```

C Collinear Lagrange points calculation

```

import numpy as np
import matplotlib.pyplot as plt
from CR3BP_solver import CR3BP_rhs, rotating_to_inertial

def dU_tilde(x, mu):
    """
    First derivative of the effective potential  $U_{\text{tilde}}(x, 0)$  with respect to  $x$ 
    in the Circular Restricted Three-Body Problem (CR3BP).
    """

```

```

Piecewise definition:
    Region I1:  $x < -\mu$ 
    Region I2:  $-\mu \leq x \leq 1 - \mu$ 
    Region I3:  $x > 1 - \mu$ 

Parameters
-----
x : float
    Position(s) along the x-axis (non-dimensional).
mu : float
    Mass parameter,  $\mu = m_2 / (m_1 + m_2)$ .

Returns
-----
dUdx : float
    Value of  $dU_{\text{tilde}}/dx$  at the given x.
"""

if x < -mu:
    return -x - (1 - mu) / (x + mu) ** 2 - mu / (x - 1 + mu) ** 2

elif x <= 1 - mu:
    return -x + (1 - mu) / (x + mu) ** 2 - mu / (x - 1 + mu) ** 2

else: # x > 1 - mu
    return -x + (1 - mu) / (x + mu) ** 2 + mu / (x - 1 + mu) ** 2

def ddU_tilde(x, mu):
    """
    Second derivative of the effective potential  $U_{\text{tilde}}(x, 0)$  w.r.t.  $x$ 
    (CR3BP), piecewise.

Parameters
-----
x : float
    Position along the x-axis (non-dimensional).
mu : float
    Mass parameter,  $\mu = m_2 / (m_1 + m_2)$ .

Returns
-----
float
    The value of  $dd_{U_{\text{tilde}}}/ddx$  at  $x$ .
"""

if x < -mu:
    return -1 + 2*(1 - mu)/(x + mu)**3 + 2*mu/(x - 1 + mu)**3

elif x <= 1 - mu:
    return -1 - 2*(1 - mu)/(x + mu)**3 + 2*mu/(x - 1 + mu)**3

else: # x > 1 - mu
    return -1 - 2*(1 - mu)/(x + mu)**3 - 2*mu/(x - 1 + mu)**3

def Newtons_method_CR3BP(x0, mu, tol = 1e-12):
    """
    Implemetation of Newtons's method for aproximation of the collinear
    Lagrange points in our Earth-Moon primary system.

Parameters

```

```

-----
x0 : float
    Startingpoint for Newton itteration.
tol : foalt, optional
    Sets the step tolerance. The default is 1e-12.

Returns
-----
x : float
    Position of the aproximated Lagrange point.

"""
# First step
x = x0
x_new = x - dU_tilde(x, mu)/ddU_tilde(x, mu)

it = 1 # Counting itterations needed to obtain a solution within tol

# Running itteraton as long as the stepsize is greater than tol
while abs(x_new - x) > tol:
    it += 1
    x = x_new
    x_new = x - dU_tilde(x, mu)/ddU_tilde(x, mu)

return x_new, it

if __name__ == '__main__':

    mu = 1.215e-2 # Earth{Moon mass parameter
    m1_pos = (-mu, 0.0) # Position of the Earth
    m2_pos = (1 - mu, 0.0) # Position of the Moon

    # Calcukating the different Lagrange points by starting within the different
    ↪ domains
    L3, it3 = Newtons_method_CR3BP(-2*mu, mu)
    L1, it1 = Newtons_method_CR3BP((1-mu)/2, mu)
    L2, it2 = Newtons_method_CR3BP(1, mu)
    print(L1, L2, L3)
    print(it1, it2, it3)

    # Plotting the Lagrange points in the rotating frame of referance
    plt.scatter(*m1_pos, s=200, marker='o', label='Earth $m_1$', color =
    ↪ 'cadetblue')
    plt.scatter(*m2_pos, s=50, marker='o', label='Moon $m_2$', color = 'grey')
    plt.scatter(L1, 0, s=10, label = '$L_1$', color = 'indianred')
    plt.scatter(L2, 0, s=10, label = '$L_2$', color = 'brown')
    plt.scatter(L3, 0, s=10, label = '$L_3$', color = 'maroon')
    plt.xlabel("$x$")
    plt.ylabel("$y$")
    plt.title("Collinear Lagrange points (CR3BP)")
    plt.legend(fontsize = 9)
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

```

D Orbit calculation for initial condition at L_1

```
import numpy as np
import matplotlib.pyplot as plt
from CR3BP_solver import CR3BP_rhs, rotating_to_inertial
from scipy.integrate import solve_ivp

# Parameters
mu      = 1.215e-2      # Earth{Moon mass parameter (example)
t_span = (0.0, 25.0)    # dimensionless time interval tau \in [0, T]
t_array = np.linspace(t_span[0], t_span[1], 5000)

L1 = 0.8369180073169304

x0, y0, z0 = L1, 0, 0
xi0, eta0, zeta0 = 0, 0, 0
Y0 = np.array([x0, y0, z0, xi0, eta0, zeta0])

# Integrate with DOP853
sol = solve_ivp(
    CR3BP_rhs, t_span, Y0, t_eval=t_array,
    method="DOP853", rtol=1e-10, atol=1e-12)

if not sol.success:
    raise RuntimeError(sol.message)

# Storing solutions for the planar motion
x = sol.y[0]
y = sol.y[1]
z = sol.y[2]

# Position of the moon
m2_pos = (1 - mu, 0.0)

# Plotting planar motion in rotating frame
plt.figure(figsize=(6,6))
plt.plot(x, y, linestyle = '--', color = 'maroon')
plt.scatter(*m2_pos, s=60, marker='o', label='Moon  $m_2$ ', color = 'grey')
plt.scatter(L1, 0, s=30, label = ' $L_1$ ', color = 'indianred')
plt.axis('equal')
plt.xlabel(" $x$ ")
plt.ylabel(" $y$ ")
plt.title("Orbit starting in  $L_1$  (Rotating frame)")
plt.legend(fontsize = 9)
plt.grid(True, alpha=0.3)
plt.show()

# Converting orbit to inertial frame
xI, yI, zI = rotating_to_inertial(t_array, x, y, z)

# Calculating the orbit of the moon
x2_orbit = (1 - mu) * np.cos(t_array)
y2_orbit = (1 - mu) * np.sin(t_array)

plt.figure(figsize=(6,6))
```

```
plt.plot(x2_orbit, y2_orbit, label='Moon  $m_2$ ', color = 'grey', linewidth=2)
plt.plot(xI, yI, linestyle = '--', color = 'maroon')
plt.axis('equal')
plt.xlabel(" $x$ ")
plt.ylabel(" $y$ ")
plt.title("Orbit starting in  $L_1$  (Inertial frame)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```